

MATERIAL DE APOYO

Guía de Git y GitHub

Programación para Ciencia de Datos

Entorno de desarrollo con Python y JupyterLab

Versión para Windows 10 y 11

Magíster en Ciencia de Datos e Inteligencia Artificial

2026

Sigue los pasos en orden. Copia los comandos tal como aparecen y léelos: entender qué hace cada uno es parte del aprendizaje. Esta guía asume que partes desde cero, sin Python ni Git instalados.

Índice

1. Conceptos previos: Git y GitHub	3
1.1. ¿Qué es Git?	3
1.2. ¿Qué es GitHub?	3
2. PARTE I. Instalación y preparación del entorno	3
2.1. Instalar Git para Windows	3
2.2. Instalar Python 3	4
2.3. Configurar Git por primera vez	4
3. PARTE II. Crear el primer proyecto y su entorno	5
3.1. Crear la carpeta del proyecto	5
3.2. Crear y activar un entorno virtual exclusivo	5
3.3. Instalar la base de trabajo dentro del entorno	6
3.4. Registrar el kernel del proyecto	6
3.5. Extensiones e integración con Git	6
3.6. Inicializar Git en el proyecto	6
3.7. Crear los archivos iniciales	7
4. PARTE III. Los tres estados de Git	7
4.1. Working Directory (directorio de trabajo)	7
4.2. Staging Area (zona de preparación)	7
4.3. Committed (confirmado en el historial)	7
5. PARTE IV. Primer flujo completo con Git	8
5.1. Revisar el estado	8
5.2. Pasar archivos al staging	8
5.3. Crear el primer commit	8
5.4. Ver el historial	8
6. PARTE V. El archivo .gitignore	8
7. PARTE VI. Notebooks y Git: higiene para ciencia de datos	9
7.1. Solución 1: limpiar salidas con nbstripout	9
7.2. Solución 2: emparejar con Jupyter	9
8. PARTE VII. Reproducibilidad del entorno	10

9. PARTE VIII. Cuenta y repositorio en GitHub	10
9.1. Crear la cuenta	10
9.2. Crear el repositorio remoto	10
10. PARTE IX. Conectar Git local con GitHub	11
10.1. Agregar el remoto	11
10.2. Verificar el remoto	11
10.3. Autenticarse con GitHub CLI	11
10.4. Subir el proyecto por primera vez	11
11. PARTE X. Flujo de trabajo normal	12
12. PARTE XI. Ramas	12
12.1. ¿Qué es una rama?	12
12.2. Crear y cambiar de rama	12
12.3. Trabajar y guardar en la rama	12
13. PARTE XII. Colaboración con Pull Requests	13
13.1. Fusión local (alternativa simple)	13
14. PARTE XIII. Errores comunes y su significado	13
14.1. fatal: unable to auto-detect email address	13
14.2. src refspec main does not match any	14
14.3. remote origin already exists	14
14.4. No se puede cargar el archivo Activate.ps1 ... scripts deshabilitados	14
14.5. 'python' no se reconoce como un comando	14
15. PARTE XIV. Buenas prácticas	14
16. PARTE XV. Secuencia consolidada desde cero	14
16.1. Del inicio a la publicación	15
16.2. Desarrollo con ramas y Pull Request	15
Conclusión	15

1 Conceptos previos: Git y GitHub

1.1 ¿Qué es Git?

Git es un sistema de control de versiones. Su función es registrar los cambios que se realizan en un proyecto a lo largo del tiempo. Gracias a Git puedes saber qué modificaste, cuándo lo hiciste y recuperar versiones anteriores si algo sale mal. En ciencia de datos esto es esencial: un experimento que funcionaba ayer y hoy no, casi siempre se resuelve mirando qué cambió en el historial.

1.2 ¿Qué es GitHub?

GitHub es una plataforma en la nube que aloja repositorios Git. Mientras Git trabaja localmente en tu computador, GitHub te permite respaldar el proyecto en internet, compartirlo, colaborar con otras personas, trabajar con ramas y centralizar el historial.

Nota

En simple:

- **Git** = control de versiones local.
- **GitHub** = repositorio remoto y plataforma de colaboración.

2 PARTE I. Instalación y preparación del entorno

2.1 Instalar Git para Windows

Descarga el instalador desde <https://git-scm.com/download/win> y ejecútalo. Puedes aceptar las opciones por defecto; lo importante es que la instalación incluye **Git Bash**, la terminal que usaremos en toda la guía.

Nota

Git Bash es una terminal que se instala junto con Git en Windows y permite ejecutar los comandos de Git y de Python de forma cómoda. Existen también **PowerShell** y el **Símbolo del sistema (cmd)**; los comandos de Git son idénticos en las tres, pero la activación del entorno virtual cambia según la terminal (lo veremos más adelante). Recomendamos Git Bash para seguir esta guía al pie de la letra.

Abre **Git Bash** (búscalo en el menú Inicio) y verifica la instalación:

```
git --version
```

Nota

Debería mostrar algo similar a `git version 2.x.x`. El número exacto no importa: basta con que el comando responda.

2.2 Instalar Python 3

Descarga Python desde el sitio oficial: <https://www.python.org/>, y ejecuta el instalador (.exe).

Atención

En la **primera pantalla** del instalador, marca la casilla “**Add python.exe to PATH**” antes de continuar. Es el error más común en Windows: si no la marcas, al escribir `python` la terminal responderá que el comando no se reconoce, y tendrás que reinstalar o configurar el PATH a mano.

Cierra y vuelve a abrir Git Bash, y verifica la instalación:

```
python --version
```

Debería mostrar algo similar a `Python 3.12.x`.

Nota

En Windows el comando suele ser `python` (a diferencia de macOS/Linux, donde es `python3`). Si `python` no funciona, prueba con el lanzador de Python: `py -version`.

Recomendación

No instales la versión más reciente de Python solo por ser la más nueva. Las librerías de ciencia de datos y *deep learning* suelen tardar meses en dar soporte completo a cada versión. A día de hoy lo más seguro para el magíster es **Python 3.12** (3.13 como máximo). Reserva las versiones más nuevas para cuando todo el ecosistema las soporte.

Nota

GPU para *deep learning* (opcional). Si tu equipo tiene una GPU NVIDIA y más adelante usarás PyTorch con aceleración, no lo instales con el `pip install torch` genérico: usa el comando específico con CUDA que indica el sitio oficial (<https://pytorch.org/get-started/locally/>). Para empezar con pandas y scikit-learn no necesitas nada de esto.

2.3 Configurar Git por primera vez

Git necesita saber quién realiza los commits. Configura tu nombre y correo, y de paso deja `main` como nombre por defecto de la rama principal en los repositorios nuevos:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
git config --global init.defaultBranch main
```

Verifica la configuración:

```
git config --global --list
```

Nota

-**global** significa que esa configuración quedará disponible para todos los repositorios de tu usuario en ese equipo. Configurar `init.defaultBranch main` evita tener que renombrar la rama más adelante.

3 PARTE II. Crear el primer proyecto y su entorno

Atención

El **orden importa**. Primero se crea la carpeta del proyecto, dentro de ella el entorno virtual, y recién entonces se instalan los paquetes *dentro* del entorno. Así nunca tocas el Python del sistema y cada proyecto queda aislado, con sus propias versiones de paquetes.

3.1 Crear la carpeta del proyecto

```
mkdir mi_app
cd mi_app
```

3.2 Crear y activar un entorno virtual exclusivo

No conviene usar un solo Python para todo: un entorno por proyecto evita conflictos entre versiones de paquetes. Crea el entorno con:

```
python -m venv .venv
```

La activación depende de la terminal que uses:

```
# Git Bash (recomendado en esta guía)
source .venv/Scripts/activate

# PowerShell
.venv\Scripts\Activate.ps1

# Símbolo del sistema (cmd)
.venv\Scripts\activate.bat
```

Fíjate que en Windows la carpeta es **Scripts** (no **bin** como en macOS/Linux). Cuando el entorno está activo, verás (`.venv`) al inicio de la línea. **A partir de aquí usas python y pip con normalidad**, porque apuntan al entorno.

Atención

Si usas **PowerShell** y al activar aparece el error “*no se puede cargar el archivo ... la ejecución de scripts está deshabilitada en este sistema*”, habilita los scripts del usuario una sola vez con:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

y vuelve a activar. Con Git Bash no necesitas este paso.

Nota

Para salir del entorno en cualquier momento: **deactivate**. Recuerda volver a activarlo (con la línea que corresponda a tu terminal) cada vez que abras una ventana nueva en el proyecto.

3.3 Instalar la base de trabajo dentro del entorno

```
python -m pip install --upgrade pip
python -m pip install jupyterlab notebook ipykernel \
    numpy pandas matplotlib seaborn scikit-learn
```

Abre JupyterLab con:

```
python -m jupyter lab
```

3.4 Registrar el kernel del proyecto

Esto permite que cada notebook use el entorno correcto. Usa un nombre coherente con el proyecto:

```
python -m ipykernel install --user --name mi_app \
    --display-name "Python (mi_app)"
```

3.5 Extensiones e integración con Git

JupyterLab se amplía con extensiones. Dos son muy útiles en ciencia de datos:

```
python -m pip install jupyterlab-git jupyterlab-text nbstripout
```

- **jupyterlab-git** añade un panel de Git dentro de JupyterLab.
- **jupyterlab-text** empareja un notebook `.ipynb` con un `.py`, lo que facilita versionar y revisar el código.
- **nbstripout** limpia las salidas de los notebooks antes de cada commit (lo configuramos en la Parte VI).

3.6 Inicializar Git en el proyecto

```
git init
```

Esto crea un repositorio Git local en la carpeta. Desde este momento Git puede rastrear los cambios.

3.7 Crear los archivos iniciales

```
touch README.md
```

Crear el archivo.ipynb

El archivo .ipynb es un notebook de Jupyter: un documento que reúne en un mismo lugar el código, los resultados que produce al ejecutarse y el texto explicativo. En JupyterLab puedes crearlo de dos formas: desde el Launcher, en la sección Notebook, o desde el menú File New Notebook. En ambos casos JupyterLab te pedirá (o usará) un kernel: elige el de tu proyecto, no el genérico "Python 3", para que el notebook quede vinculado al entorno virtual donde instalaste tus librerías. Luego solo te queda renombrarlo y guardarlo para que aparezca en la carpeta del proyecto

El .gitignore lo crearemos con contenido en la Parte V, *antes* del primer commit.

4 PARTE III. Los tres estados de Git

Git trabaja con tres estados fundamentales. Entenderlos bien es la base de todo.

4.1 Working Directory (directorio de trabajo)

Es donde creas, editas y guardas archivos. Git todavía no los ha preparado ni guardado en el historial. Si modificas `app.ipynb` y ejecutas `git status`, Git detectará el cambio, pero el archivo sigue solo en el área de trabajo.

4.2 Staging Area (zona de preparación)

Es la zona intermedia donde se colocan los cambios que quieres incluir en el próximo commit. Se envían archivos con:

```
git add nombre_archivo
git add .
```

Nota

Un **commit** es un registro guardado de cambios: una fotografía del proyecto en un momento concreto. Git deja constancia de qué cambió, cuándo, con qué mensaje y quién lo hizo.

4.3 Committed (confirmado en el historial)

Es el estado en que Git ya guardó esa versión en el historial:

```
git commit -m "mensaje del commit"
```


5 PARTE IV. Primer flujo completo con Git

5.1 Revisar el estado

```
git status
```

Es el comando que más vas a usar: muestra archivos nuevos, modificados y preparados.

5.2 Pasar archivos al staging

```
git add .
```

5.3 Crear el primer commit

```
git commit -m "Crea estructura inicial del proyecto"
```

Recomendación

El mensaje debe describir el cambio con claridad. Ejemplos:

```
git commit -m "Agrega carga y limpieza del dataset"
git commit -m "Implementa modelo base de regresion"
git commit -m "Corrige fuga de datos en el split"
```

5.4 Ver el historial

```
git log
git log --oneline
```

6 PARTE V. El archivo .gitignore

Es un archivo donde indicas qué elementos Git **no** debe versionar: entornos, caché, datos pesados y archivos sensibles. Créalo con este contenido típico para un proyecto de ciencia de datos en Windows:

```
# Entorno virtual
.venv/

# Cache de Python
__pycache__/
*.pyc
```

```
# Jupyter
.ipynb_checkpoints/

# Windows
Thumbs.db
desktop.ini

# Variables de entorno y secretos
.env

# Datos y modelos pesados
data/raw/
*.ckpt
*.pth
```

Atención

Crea y completa el `.gitignore` **antes** de tu primer `git add ..` Si no, corres el riesgo de versionar por error la carpeta `.venv/` (cientos de archivos) o un `.env` con credenciales. Los archivos `Thumbs.db` y `desktop.ini` los genera Windows automáticamente en las carpetas; ignorarlos mantiene el repositorio limpio.

7 PARTE VI. Notebooks y Git: higiene para ciencia de datos

Este es el punto que más problemas causa y que las guías genéricas omiten. Un archivo `.ipynb` guarda, dentro del mismo JSON, las **salidas** de cada celda y los **contadores de ejecución**. Eso provoca tres problemas:

- **Diffs ilegibles:** cualquier cambio mínimo genera diferencias enormes.
- **Conflictos de merge** constantes y difíciles de resolver.
- **Filtración de datos:** una tabla o un gráfico impreso puede dejar información sensible dentro del repositorio.

7.1 Solución 1: limpiar salidas con nbstripout

Ya instalado `nbstripout`, actívalo en el repositorio una sola vez:

```
nbstripout --install
```

A partir de ese momento, cada vez que hagas `git add` de un notebook, Git guardará la versión *sin* salidas. Tu notebook en pantalla no se altera; solo se limpia lo que va al historial.

7.2 Solución 2: emparejar con Jupyter

Como alternativa (o complemento), puedes vincular cada notebook a un archivo `.py` de texto plano, versionar el `.py` y dejar el `.ipynb` fuera de Git. Desde la terminal:

```
jupyter --set-formats ipynb,py:percent app.ipynb
```

Esto crea `app.py` sincronizado con el notebook. El `.py` se lee y se revisa mucho mejor en un *Pull Request*.

Recomendación

Para empezar, con `nbstripout` es suficiente. Cuando trabajes en grupo o quieras revisiones de código serias, añade el emparejamiento con JupyterText.

8 PARTE VII. Reproducibilidad del entorno

En un contexto científico, otra persona (o quien corrija tu trabajo) debe poder recrear tu entorno de forma idéntica. Por eso, cada vez que instales o actualices paquetes, guarda la lista exacta:

```
python -m pip freeze > requirements.txt
```

Versiona el `requirements.txt`. Cualquiera podrá reconstruir el entorno con:

```
python -m venv .venv
source .venv/Scripts/activate
python -m pip install -r requirements.txt
```

Atención

Cuidado al generar `requirements.txt` desde Anaconda. Si tu Python viene de Anaconda/conda, `pip freeze` puede escribir paquetes con rutas locales (`paquete @ file:///C:/...`) que fallan al instalar en otro equipo con *"No such file or directory"*. Para evitarlo, genera la lista con `python -m pip list --format=freeze > requirements.txt`, o escribe a mano solo los paquetes que tu proyecto usa.

Nota

Más adelante encontrarás herramientas modernas (`uv`, `pypi`, `Poetry`) que generan un *lockfile* con versiones exactas y resuelven dependencias más rápido. Para iniciarse, `requirements.txt` cumple perfectamente.

9 PARTE VIII. Cuenta y repositorio en GitHub

9.1 Crear la cuenta

Entra a <https://github.com>, regístrate, confirma tu correo e inicia sesión.

Recomendación

Usa un nombre de usuario profesional: tus repositorios pueden formar parte de tu portafolio.

9.2 Crear el repositorio remoto

Dentro de GitHub: haz clic en **New repository**, asígnale un nombre, elige si será público o privado y créalo. No marques la opción de añadir README ni `.gitignore` si ya los tienes en local

(evita conflictos en el primer push).

10 PARTE IX. Conectar Git local con GitHub

10.1 Agregar el remoto

```
git remote add origin https://github.com/USUARIO/mi_app.git
```

`origin` es el nombre convencional del repositorio remoto principal.

10.2 Verificar el remoto

```
git remote -v
```

10.3 Autenticarse con GitHub CLI

Instala la CLI si no la tienes (en Windows, con winget) y autenticáte:

```
winget install --id GitHub.cli  
gh auth login
```

Cuando te pregunte, responde:

- Where do you use GitHub? → GitHub.com
- Preferred protocol for Git operations? → HTTPS
- Authenticate Git with your GitHub credentials? → Yes
- How would you like to authenticate? → Login with a web browser

Se abrirá el navegador. Inicia sesión, autoriza y vuelve a la terminal. Verifica con:

```
gh auth status
```

10.4 Subir el proyecto por primera vez

```
git push -u origin main
```

Nota

`-u` deja enlazada la rama local con la remota. Después bastará con `git push`. Si GitHub pide autenticación durante el push, inicia sesión en el navegador, autoriza y espera a que termine.

11 PARTE X. Flujo de trabajo normal

Cada vez que avances en el proyecto:

```
git status
git add .
git commit -m "Describe el avance"
git push
```

Nota

Ese es el ciclo central: **status** (revisar) → **add** (preparar) → **commit** (registrar) → **push** (subir).

12 PARTE XI. Ramas

12.1 ¿Qué es una rama?

Una rama es una línea de trabajo independiente dentro del mismo proyecto. Sirve para desarrollar funcionalidades, corregir errores o experimentar sin afectar la versión estable. **main** representa esa versión estable.

12.2 Crear y cambiar de rama

Forma moderna y recomendada (crear y cambiar en un paso):

```
git switch -c experimento_modelo
```

Otros comandos útiles:

```
git branch          # ver ramas (la actual con *)
git switch main      # volver a la rama principal
```

Nota

Verás también **git checkout** en muchos tutoriales: hace lo mismo, pero **git switch** y **git restore** son la forma moderna y más clara.

12.3 Trabajar y guardar en la rama

```
git add .
git commit -m "Agrega experimento de modelo"
```

13 PARTE XII. Colaboración con Pull Requests

En GitHub, la forma correcta de integrar y **revisar** cambios no es fusionar a ciegas en local, sino el **Pull Request (PR)**. Es el mecanismo que usarás cuando trabajes en grupo o cuando un docente deba revisar tu avance.

```
# 1. Crear y trabajar en una rama
git switch -c experimento_modelo
# ... editar archivos ...
git add .
git commit -m "Agrega experimento de modelo"

# 2. Subir la rama al remoto
git push -u origin experimento_modelo
```

3. En GitHub aparecerá un botón para abrir un **Pull Request** desde esa rama hacia **main**. Ábrelo, describe el cambio y solicita revisión.

4. El revisor comenta línea por línea. Cuando todo esté conforme, se fusiona el PR desde la propia interfaz de GitHub.

5. Actualiza tu **main** local:

```
git switch main
git pull
git branch -d experimento_modelo
```

Recomendación

El PR deja un registro de la discusión y la revisión, no solo del código. Es el estándar profesional y lo que esperan ver en cualquier equipo de datos.

13.1 Fusión local (alternativa simple)

Para proyectos individuales y pequeños, también puedes fusionar en local:

```
git switch main
git merge experimento_modelo
git push
git branch -d experimento_modelo
```

14 PARTE XIII. Errores comunes y su significado

14.1 fatal: unable to auto-detect email address

Git no tiene configurado el nombre o el correo.

```
git config --global user.name "Nombre"
git config --global user.email "correo@ejemplo.com"
```

14.2 src refspec main does not match any

Todavía no existe ningún commit en **main**. Haz un commit y luego sube:

```
git add .  
git commit -m "Primer commit"  
git push -u origin main
```

14.3 remote origin already exists

Ya hay un remoto llamado **origin**. Cambia la URL:

```
git remote set-url origin NUEVA_URL
```

14.4 No se puede cargar el archivo Activate.ps1 ... scripts deshabilitados

Aparece al activar el entorno en **PowerShell**. Habilita los scripts del usuario una sola vez y vuelve a activar:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Con Git Bash no ocurre este error.

14.5 'python' no se reconoce como un comando

No marcaste “**Add python.exe to PATH**” al instalar Python. Reinstálalo marcando esa casilla, o prueba con el lanzador **py** en lugar de **python**.

15 PARTE XIV. Buenas prácticas

- **Commits pequeños y claros.** Mejor varios commits acotados que uno enorme.
- **Mensajes descriptivos.** El mensaje debe explicar el cambio.
- **Revisa siempre con `git status`** antes de `add`, `commit` o `merge`.
- **No trabajes siempre en `main`.** Usa ramas para los cambios importantes.
- **Sube avances con frecuencia** para no perder trabajo.
- **Mantén un `README.md`** que explique qué hace el proyecto, cómo se ejecuta, qué tecnologías usa y quién lo desarrolló.
- **Limpia las salidas de los notebooks** (`nbstripout`) y **actualiza el `requirements.txt`** cuando cambien las dependencias.

16 PARTE XV. Secuencia consolidada desde cero

16.1 Del inicio a la publicación

```
# Configuración (una sola vez por equipo)
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
git config --global init.defaultBranch main

# Crear proyecto y entorno (en Git Bash)
mkdir mi_app && cd mi_app
python -m venv .venv
source .venv/Scripts/activate
python -m pip install --upgrade pip
python -m pip install jupyterlab numpy pandas matplotlib \
    scikit-learn jupyter nbstripout

# Inicializar Git y limpiar notebooks
git init
nbstripout --install

# Crear archivos (incluido .gitignore CON contenido)
touch README.md app.ipynb
# ... escribir el .gitignore ...

# Guardar dependencias y primer commit
python -m pip freeze > requirements.txt
git add .
git commit -m "Crea estructura inicial del proyecto"

# Conectar con GitHub y subir
git remote add origin https://github.com/USUARIO/mi_app.git
git push -u origin main
```

16.2 Desarrollo con ramas y Pull Request

```
git switch -c nueva_funcionalidad
# ... trabajar ...
git add .
git commit -m "Agrega nueva funcionalidad"
git push -u origin nueva_funcionalidad
# Abrir el Pull Request en GitHub, revisar y fusionar
git switch main
git pull
git branch -d nueva_funcionalidad
```

Conclusión

Git y GitHub no son solo herramientas técnicas: son parte del modo profesional de desarrollar y de investigar. Aportan orden, trazabilidad y trabajo colaborativo. En ciencia de datos suman, además, dos cosas que distinguen un trabajo serio: **reproducibilidad** (cualquiera puede recrear tu entorno y tus resultados) e **higiene de notebooks** (un historial limpio, sin salidas pesadas)

ni datos filtrados).

Nota

La idea central que debes recordar:

trabajar → revisar con `git status` → preparar con `git add` → guardar con `git commit` → subir con `git push`.

Y cuando el proyecto crece: crear rama → trabajar → commitear → subir rama → abrir Pull Request → revisar → fusionar.